

⚠ Please do not reach out to reviewers directly; messaging them will not speed up your review.

Creating Docker Environment

This page is the practical setup guide for task environments: where the Dockerfile lives, what a starter Dockerfile looks like, how compose files are shaped, and how to test the container locally.

For the canonical Dockerfile policy and detailed examples for each image check, use [Dockerfile & Image Best Practices](#). This page only summarizes the minimum requirements so you can get an environment working.

Basic Dockerfile

Your Dockerfile should be located at `environment/Dockerfile`:

DOCKERFILE

 Copy

```
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256-

WORKDIR /app

# Install system dependencies
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        asciinema \
        ca-certificates \
        git \
        tmux \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies (PIN VERSIONS!)
RUN pip install --no-cache-dir \
    numpy==1.26.4 \
    pandas==2.1.0 \
```

```
requests==2.31.0

# Copy task files
COPY app/ /app/

# Set up environment
ENV PYTHONPATH=/app
```

Minimum Checklist

Before moving on to task-specific setup, make sure your environment satisfies these basics:

- `environment/Dockerfile` exists and builds.
- The final runtime image includes `tmux` and `asciinema`.
- Every `FROM` image, and every pulled compose `image:`, includes `@sha256:<digest>`.
- The final runtime base image is sanctioned or explicitly exempt.
- Language dependencies are exact-pinned or locked.
- Apt installs use `--no-install-recommends` and clean `/var/lib/apt/lists/*` in the same layer.
- `environment/` is at most 100 MiB total and no file is over 50 MiB.
- Non-trivial environments include `.dockerignore`.
- The image does not copy `solution/`, `tests/`, or hidden verifier assets.
- `tests/test.sh` dependencies are baked into the image; the verifier does not install or download at runtime.
- Compose files do not use privileged containers or unsafe capabilities.

See [Dockerfile & Image Best Practices](#) for the full rationale and examples behind each item.

docker-compose.yaml

YAML

 Copy

```
version: "3.8"
services:
  task:
    build: .
    working_dir: /app
    volumes:
      - ./workspace:/workspace
    environment:
      - PYTHONPATH=/app
```

Multi-Container Setup

For tasks requiring multiple services:

YAML

 Copy

```
version: "3.8"
services:
  app:
    build: .
    depends_on:
      - db
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/app

  db:
    # Pin service images by digest when using image:
    image: postgres:15@sha256:<digest>
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=app
```

Common Patterns

Use digest-pinned base images in each pattern, same as in the [basic Dockerfile](#). These examples are starting points; for layer ordering, [.dockerignore](#), reproducible downloads, multi-stage builds, and runtime-image hygiene, see [Dockerfile & Image Best Practices](#).

Python Project

DOCKERFILE

 Copy

```
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256-
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ /app/src/
ENV PYTHONPATH=/app
```

Node.js Project

DOCKERFILE

 Copy

```
FROM public.ecr.aws/docker/library/node:22-bookworm-slim@sha256:<di
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY src/ /app/src/
```

System Administration Task

DOCKERFILE

 Copy

```
FROM public.ecr.aws/docker/library/ubuntu:24.04@sha256:<digest>
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        curl \
        nginx \
        vim \
    && rm -rf /var/lib/apt/lists/*
```

Git Repository Task

DOCKERFILE

 Copy

```
FROM public.ecr.aws/docker/library/python:3.13-slim-bookworm@sha256
RUN apt-get update \
    && apt-get install -y --no-install-recommends git \
    && rm -rf /var/lib/apt/lists/*

# Clone at specific commit to prevent cheating
RUN git clone https://github.com/example/repo.git /app \
    && cd /app && git checkout abc123def
```

Important: When cloning repos, pin to a specific commit so agents can't see newer commits with solutions.

Troubleshooting

Build Fails

BASH

 Copy

```
# Test your Dockerfile locally
cd environment
docker build -t my-task .
docker run -it my-task bash
```

Container Won't Start

Check docker-compose logs:

BASH

 Copy

```
docker-compose logs
```

Permission Issues

On macOS, ensure Docker Desktop settings allow socket access:

1. Settings → Advanced
2. Enable "Allow the default Docker socket to be used"

CI Checks

For the complete list of blocking and warning checks, see [CI Checks Reference](#). For Docker-specific fixes, see [Dockerfile & Image Best Practices](#).

Next Steps

- [Dockerfile & Image Best Practices](#) — Advanced requirements for reproducible, auditable, and complete images
- [Write oracle solution](#)
- [Write tests](#)

PREVIOUS

[Running Your Task](#)

NEXT

[Dockerfile Requirements](#)